

Zou_lab_control.neutral_atom 硬件接入教程

qCMOS / FPGA sequencer / frontend live plotting quickstart

从 virtual 离线闭环到真实双 PC 触发架构的第一版实验线路

2026-06-05

Contents

1	目标和边界	1
1.1	硬件闭环.	1
1.2	默认 address-switch 映射.	1
2	FPGA/Vivado 电脑	3
2.1	安装	3
2.2	build 和 program	3
2.3	启动 server	4
3	Pulse GUI.	6
3.1	打开 GUI	6
3.2	GUI 显示和硬件宽度	6
3.3	On Pulse 行为.	7
4	Scan 和 analog bus	9
4.1	named scan table	9
4.2	10-bit bus.	10
5	控制电脑 notebook.	12
5.1	连接设备.	12
5.2	绑定 preset 做 readout scan.	13
6	图和预期结果	14
6.1	Pulse preview	14
6.2	raw capture.	15
6.3	sitemap.	15
6.4	threshold.	15
6.5	detect	15
6.6	detection-time scan	15
7	常见问题	18
7.1	pulse 长度是设定两倍	18
7.2	On Pulse 没报错但示波器无信号	18
7.3	GUI 只显示少数 channel	18

7.4	scan table 不通过	18
-----	--------------------------	----

1

目标和边界

1.1 硬件闭环

`Zou_lab_control.neutral_atom` 当前推荐的真实硬件闭环是:

Experiment path

```
control computer
-> Jupyter / Pulse GUI
-> RemoteSequencer
-> FPGA/Vivado computer sequencer_server
-> Vivado/VIO runtime pulse-streamer
-> qCMOS external trigger and TTL outputs
```

GUI 只是 pulse 前端。它可以只显示 `trap/cooling/probe/emCCD` 这几路，也可以显示全部 XDC 推断 channel; 上传时仍然按 server 的完整 hardware channel order 编译。未显示、未配置或隐藏的 channel 都会补成 off bit。

1.2 默认 address-switch 映射

默认 real-hardware 路线使用历史 address-switch XDC:

Default XDC

```
references\source_archives\address_switch\address_switch.srcs\constrs_1\n_
↪ ew\addre.xdc
```

runtime wrapper 把 XDC 输出映射成 `ch00...` 的硬件 bit order。相机成像 preset 的关键映射是:

Camera-imaging subset

```
ch09 -> trap      -> M17
ch00 -> cooling    -> F15
ch03 -> probe     -> N15
```

```
ch11 -> emCCD    -> M13
```

这份 XDC 也保留 `ch06 → trig → R17`。当前 camera/qCMOS preset 使用 `ch11/emCCD/M13` 作为触发输出，不需要改 pin 映射。

默认 FPGA clock 是 50 MHz，minimal step 是 20 ns。如果示波器看到 pulse 长度是设定值两倍，优先检查 control/server/GUI 是否仍然沿用旧的 100 MHz 假设。

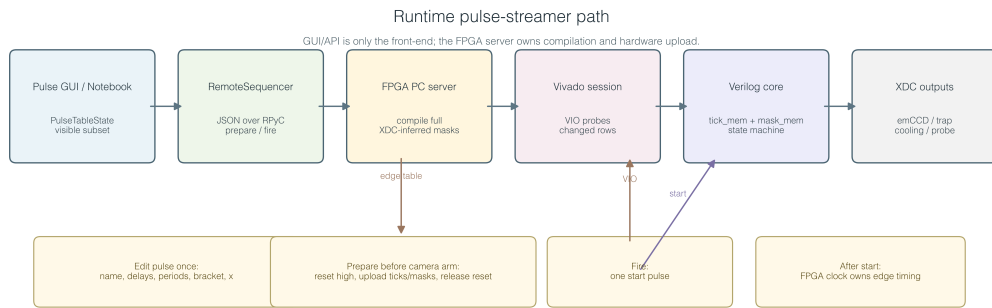


Figure 1.1: Pulse GUI 或 notebook 只负责生成 pulse state; FPGA/Vivado 电脑负责编译、上传和 fire。

2

FPGA/Vivado 电脑

2.1 安装

在 Verilog/FPGA 电脑和控制电脑上都安装 Python package:

PowerShell

```
cd D:\ZLC
install_requirements.bat
```

`install_requirements.bat` 会记录当前 Python 到 `.zlc_python_path`。之后 `pulse_gui.bat`、`fpga\run_server.bat` 和 tutorial launcher 会优先使用同一个解释器。

Vivado 只需要装在 Verilog/FPGA 电脑。若不在默认路径，设置：

PowerShell

```
$env:ZLC_PS_VIVADO_BIN = "C:\Xilinx\Vivado\2019.1\bin\vivado.bat"
```

2.2 build 和 program

推荐把 repo 放在短路径，例如 `D:\ZLC`：

PowerShell

```
cd D:\ZLC
.\fpga\build_and_program.bat --help
.\fpga\build_and_program.bat --check
.\fpga\build_and_program.bat
```

`-check` 做 HDL/VIO 宽度和容量自查，不需要 program 板卡。`build_and_program.bat` 默认工程位置是：

Vivado outputs

```
fpga\build\address_switch\address_switch.xpr
fpga\build\address_switch\address_switch.runs\impl_1\zlc_pulse_streamer_top_
↪ op_address_switch.bit
fpga\build\address_switch\address_switch.runs\impl_1\zlc_pulse_streamer_top_
↪ op_address_switch.ltx
```

如果使用另一份已确认的板级 XDC:

PowerShell

```
$env:ZLC_PS_XDC = "D:\fpga_pin_maps\my_address_switch_board.xdc"
.\fpga\build_and_program.bat
```

Vivado GUI 路线等价:

1. 打开 Vivado 2019.x。
2. 如果工程还没有生成, 先运行 `fpga\build_and_program.bat -build-only`。
3. 打开 `fpga\build\address_switch\address_switch.xpr`。
4. 运行 Synthesis、Implementation、Generate Bitstream。
5. 进入 Hardware Manager, Open Target, Auto Connect。
6. Program Device, 选择同一 implementation 目录下的 `zlc_pulse_streamer_top_address_switch.bit` 和 `zlc_pulse_streamer_top_address_switch.ltx`。

如果 Vivado GUI 只看到 Digilent target 但没有 FPGA device, 运行:

PowerShell

```
.\fpga\build_and_program.bat --diagnose
```

然后检查板卡供电、JTAG/mode jumper、线缆、power-source jumper 和 Hardware Manager Auto Connect。

2.3 启动 server

program 完 FPGA 后, 在 Verilog/FPGA 电脑上启动 server:

PowerShell

```
cd D:\ZLC
.\fpga\run_server.bat --check-config
.\fpga\run_server.bat
```

`-check-config` 会打印 project、bit、ltx、XDC、channel list、trigger channel、clock、edge capacity 和 scan capacity 后退出。确认无误后再运行长驻 server。

默认 server 设置:

Server defaults

```
host:          0.0.0.0
port:          18861
backend:       vivado-session
```

```
clock:          50 MHz
step:           20 ns
channels:       inferred from XDC, fallback ch00 ... ch61
trigger channel: inferred from XDC label "emCCD", normally ch11
```

server 不单独定义另一个 trigger 名字。它从 XDC label 推断硬件 channel, 例如 [emCCD](#) → [ch11](#), 再把这个硬件 bit 传给 trigger counting 和 camera frame normalization。

3

Pulse GUI

3.1 打开 GUI

在 Verilog/FPGA 电脑本机:

PowerShell

```
.\pulse_gui.bat --remote-host 127.0.0.1 --remote-port 18861 --state  
↪ .\pulses\camera_imaging_address_switch.json
```

在控制电脑上, 把 `-remote-host` 改成 Verilog/FPGA 电脑 IP:

PowerShell

```
.\pulse_gui.bat --remote-host 192.168.0.20 --remote-port 18861 --state  
↪ .\pulses\camera_imaging_address_switch.json
```

离线编辑 pulse JSON:

PowerShell

```
.\pulse_gui.bat --no-sequencer --state  
↪ .\pulses\camera_imaging_address_switch.json
```

3.2 GUI 显示和硬件宽度

`pulses\camera _imaging_address_switch.json` 保存完整硬件 channel order, 但默认只显示:

Visible subset

```
ch09 trap  
ch00 cooling  
ch03 probe  
ch11 emCCD
```

`Add Channel` 可以把隐藏的 channel 加回来; `Hide Off` 只隐藏全程 off 的 channel; `Show All`

显示全部硬件 channel。隐藏只是前端视图，上传仍然按 full-width channel order 编译。Name 面板左侧 raw column 在能读到 XDC 时显示物理 package pin，例如 M17/F15/N15/M13；chNN 仍然保存在 JSON/API state 和 tooltip 中。

Edit 页可以按区域理解：

Channel Names and Duration pulse 名称、总时长、period 数、visible count 和 display label。左侧 raw column 显示物理 pin，右侧 name 是实验可读名。

Delay and Scan Step、active scan params、scan file 和每个 visible channel 的 delay。Step 来自当前 sequencer clock，默认 20 ns。

period cards 每张卡是一段持续时间。duration/unit 在上方；下面每行 checkbox 对应该 channel 在该 period 是否输出高电平。

Control Buttons On Pulse/Stop Pulse、增删 period、添加 repeat bracket、保存/读取 JSON。

Channel View 从 hidden list 添加 channel、隐藏全程 off 的 channel、显示全部 channel。

Preview 页不需要手动 refresh。打开 Preview 时 GUI 会用当前未展开的 `PulseTableState` 调用 `zf.plot(..., kind="pulse")`。默认只画 active channel；打开 **Show off rows** 会画完整 channel list。y 轴显示 display label，不再显示总标题 **Pulse**。repeat 会用 $\times\infty$ 或 $\times N$ 标在 bracket 上；包含 named scan parameter 的 duration 或 delay 会用贯穿所有 channel 的半透明区域标出来。

3.3 On Pulse 行为

按 On Pulse 时，GUI 会：

1. 从当前 period-card state 生成 `PulseTableState`。
2. 按 attached sequencer 的 clock 和 channel list 编译，校验 20 ns 对齐。
3. 把 missing channel 补成 off bit。
4. 通过 `RemoteSequencer.prepare(...)` 上传 edge table。
5. 调用 `fire()` 给 FPGA 一个 start pulse。

Stop Pulse 调用 sequencer safe/reset。GUI 没有独立 sync 按钮；GUI 也不是单独硬件控制层。

默认 camera preset 在 50 MHz 下编译为：

Expected compile

```
ticks: 0, 100000, 105000, 106000, 1105000, 1106000
masks: 513, 512, 2568, 520, 512, 0
trigger_count: 1
repeat_forever: true
```

$513 = \text{ch00} + \text{ch09}$, $2568 = \text{ch03} + \text{ch09} + \text{ch11}$ 。如果示波器上没有输出，先确认观察的是这些 XDC 物理端口对应的线，而不是旧四路假设。

API 等价操作是：

Python

```
state =
↳ na.PulseTableState.load("pulses/camera_imaging_address_switch.json")
program = state.compile(
```

```
clock_hz=exp.devices.sequencer.clock_hz,  
trigger_channels=exp.devices.sequencer.trigger_channels,  
repeat_forever=False,  
)  
exp.devices.sequencer.prepare(program)  
exp.devices.sequencer.fire()
```

`repeat_forever=True` 适合示波器 first-light, 因为输出会一直循环, 便于找线和触发。真实相机采集 finite stack 时, 应该让 readout/camera helper 先 arm camera, 再生成刚好需要帧数的 finite trigger sequence。不要让 qCMOS 等一个没有帧数边界的自由循环 pulse。

4

Scan 和 analog bus

4.1 named scan table

GUI/API 使用一个 named scan table file:

Scan table

```
# vars: pulse_width_ns(ns), trig_delay(ns)
1000 0
2000 20
4000 40
```

所有 timing 值单位是 ns, 并使用和 duration/delay 相同的 Fluent line-edit resolution 逻辑对齐到 20 ns. period duration 或 delay 里可以写 `pulse_width_ns`、`trig_delay`、`100000-pulse_width_ns` 这类表达式。Preview 不展开 scan table, 而是在对应时间段用贯穿所有 channel 的半透明标记写出表达式; FPGA 上传的是 edge template 加打包后的 scan RAM。当前 Artix-7 bitstream 最多接受两个 active timing 参数; 静态 analog/DAC edge value 可以通过每个 scan row 的 packed bus-value RAM 扫描, ramp-mode DA scan 仍需按每个 scan row 分开 prepare。

scan template 的限制是边沿顺序必须在所有 scan row 上保持不变。比如一个 period 是 `pulse_width_ns`, 另一个 delay 是 `100000-pulse_width_ns`, 当参数改变到某些点时, 如果两条边的先后顺序会交换, 单个模板就不能安全描述整组 scan。此时应拆成多个 scan chunk, 或每个点单独 prepare。这个限制不是 GUI 限制, 而是为了避免 FPGA 在一个固定 edge row 顺序里执行互相矛盾的 timing。

如果 scan 点太多导致超过单个 FPGA program 的 `MAX_SCAN_POINTS`, 优先用 API 分 chunk 连续执行, 例如 `pulse.on_pulse(wait=True, repeat_forever=False)`; 只有确实需要一次 start 自主跑完更多点时, 才重新 build 更大的 profile。build 脚本使用 `ZLC_PS_RESOURCE_TARGET_PCT` 作为资源目标, 默认 70%:

PowerShell

```
$env:ZLC_PS_RESOURCE_TARGET_PCT = "70"
$env:ZLC_PS_MAX_EDGES = "512"
$env:ZLC_PS_MAX_SCAN_POINTS = "256"
.\fpga\build_and_program.bat --check
```

4.2 10-bit bus

address-switch XDC 中的 `da_dipole[0..9]`、`da_bias_x/y/z[0..9]` 是 10-bit TTL bus。GUI/API 可以把它们折叠成逻辑 analog channel，数值范围是 `0..1023`：

- **edge**: 在当前 period 开头跳到输入值。
- **ramp**: 从上一个有数值的 period 线性过渡到输入值；FPGA bus engine 在本地生成 10-bit stair-step。
- **hold**: 沿用当前值或正在进行的 ramp，不显示数值输入。

Preview 对 bus 使用一条贴着当前值的空心阶梯线，不使用实心 TTL block、额外顶线或 `0..1024` 文字。当前 runtime 已经把 digital edge table 和 analog bus segment table 分开：TTL 输出继续上传 `tick/mask`，DA bus 上传 `bus_id/start_tick/stop_tick/start_value/stop_value/mode`，因此长 bus ramp 不再消耗普通 TTL edge rows。

Ordinary edge table vs analog bus segment table

The two tables are uploaded together but executed by different runtime logic.

Digital TTL edge table

tick	mask[61:0]
0	trap=1, cooling=1, ...
100	probe=1, emCCD=1, ...
22000000	all off

Good for sparse TTL changes.
Every row rewrites the full mask.

Analog/DA bus segment table

bus	start	stop	v0	v1	mode
0	0	0	350	350	edge
0	100	50000	350	720	ramp
2	0	0	512	512	edge

Good for DA values and ramps.
Ramp shape is generated on FPGA.

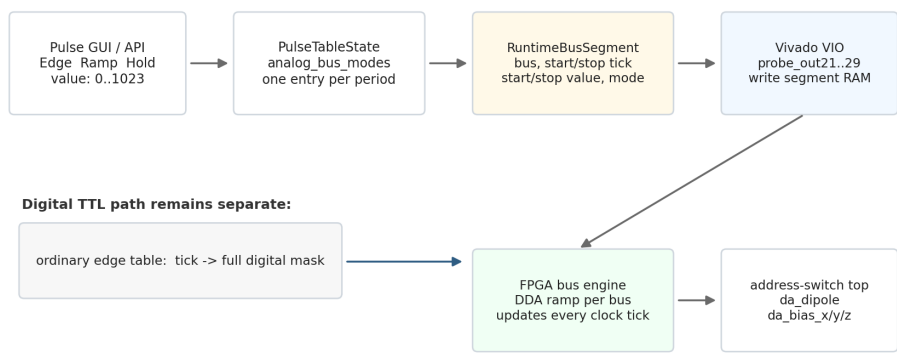
Figure 4.1: 硬件上传时的两张表。左边的普通 edge table 适合 sparse TTL；右边的 bus segment table 适合 DA edge 和 ramp。

执行顺序可以这样理解：**Prepare** 时 server 先上传普通 edge rows，再上传 bus segment rows，并把每个 bus 的有效 segment 数写进 `bus_counts`。没有使用的 bus count 为 0，所以旧 RAM 内容不会执行。**On Pulse** 触发后，普通 TTL mask 按 edge tick 更新；DA bus engine 每个 FPGA clock tick 检查当前 segment，遇到 **edge** 直接换值，遇到 **ramp** 用 DDA accumulator 生成 stair-step。硬件 scan array 支持每个 scan row 一个静态 edge-mode DA 值，但暂时不能和 analog ramp segment 同一次上传；需要扫描 analog ramp 时，按 scan point 逐个 prepare/fire。

当前 bus value 输入框是 line edit，不是 spinbox。输入超过范围时，GUI 会 clamp 到 bus 位宽允许的最大值；10-bit bus 是 `0..1023`。这比 spinbox 更适合以后支持表达式、复制粘贴和 notebook/API 生成的数值，同时仍然能在 GUI 端做范围检查。

Analog/DA bus segment path

GUI/API edits logical 10-bit buses; FPGA stores a compact segment table and generates ramps locally.



Key point: one long DA ramp consumes one bus segment, not hundreds of ordinary TTL edge rows.

Figure 4.2: analog bus 运行路径。Pulse GUI/API 编辑逻辑值，server 通过 VIO 写 segment RAM，FPGA bus engine 输出 da_dipole/da_bias_x/y/z。

5

控制电脑 notebook

5.1 连接设备

打开:

PowerShell

```
jupyter lab tutorials\nneutral_atom_hardware_quickstart.ipynb
```

连接 Verilog/FPGA 电脑:

Python

```
exp = na.connect(  
    "remote_template",  
    sequencer={"host": "192.168.0.20", "port": 18861},  
    open_devices=True,  
)
```

然后运行:

Python

```
exp.timing.configure_imaging(exposure=2e-3, load=True)  
preflight = exp.timing.preflight()  
preflight.raise_if_failed()  
  
capture = exp.camera.capture(frames=1)  
sitemap = exp.readout.sitemap(frames=20, grid_shape=(5, 7))  
threshold = exp.readout.thresholds(frames=120, site=0)  
shot = exp.readout.detect()
```

5.2 绑定 preset 做 readout scan

Python

```
pulse = exp.timing.bind_pulse("pulses/camera_imaging_address_switch.json")
pulse.set_variable("camera_exposure_ns", 2_000_000)
pulse.on_pulse(wait=True, repeat_forever=False)

times = np.linspace(0.2e-3, 8e-3, 40)
scan = exp.readout.detection_time(times, shots=30, pulse=pulse,
    ↪ live=False)
```

camera acquisition 会先 arm camera, 再 fire sequencer。多帧 acquisition 会把单 trigger sequence 转成刚好 N 个触发的有限 sequence, 避免相机等待缺失 frame。

6

图和预期结果

6.1 Pulse preview

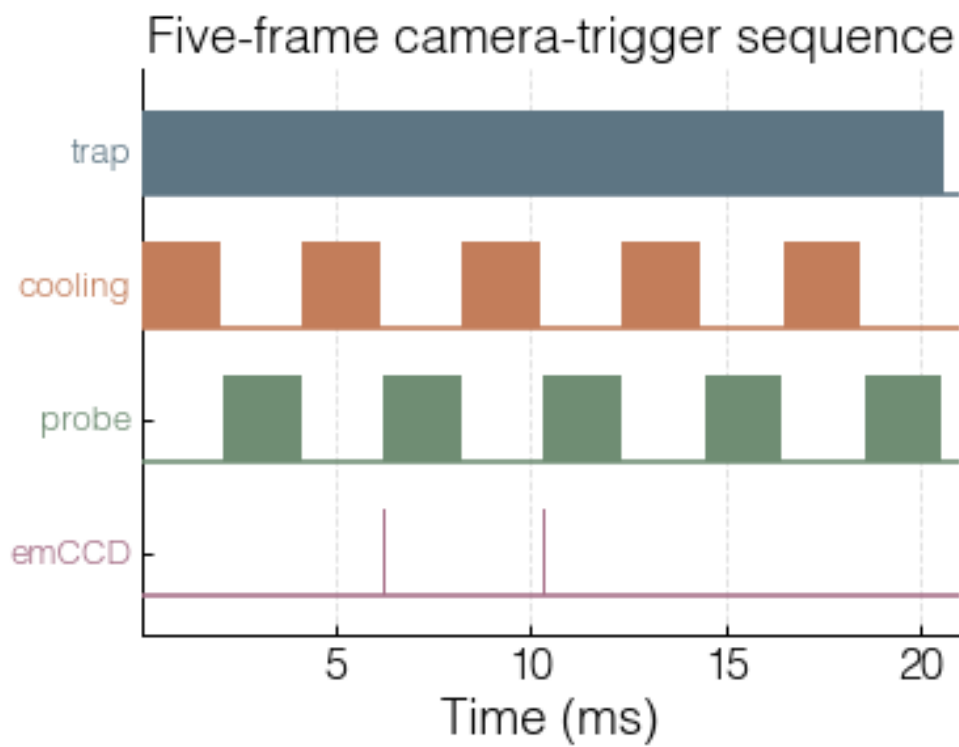


Figure 6.1: 单 shot imaging template 被 repeat 成五个 camera trigger.

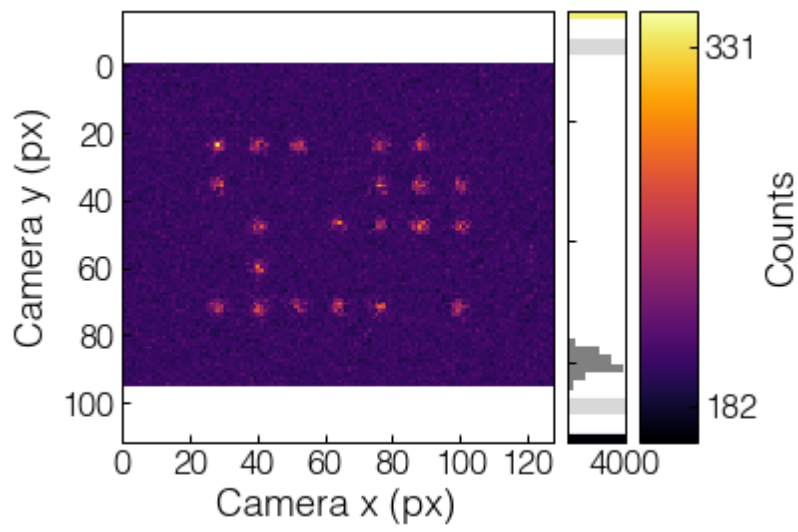


Figure 6.2: capture 只显示 raw camera frame.

6.2 raw capture

6.3 sitemap

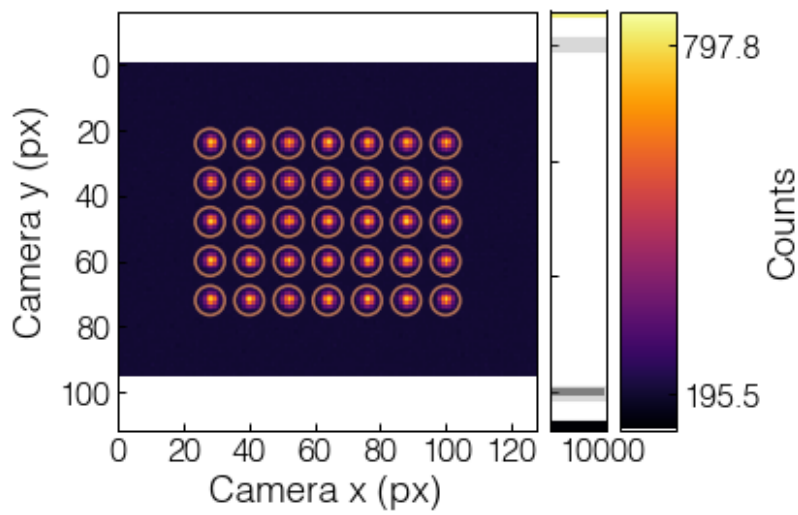


Figure 6.3: sitemap calibration 负责 site centers 和 ROI.

6.4 threshold

6.5 detect

6.6 detection-time scan

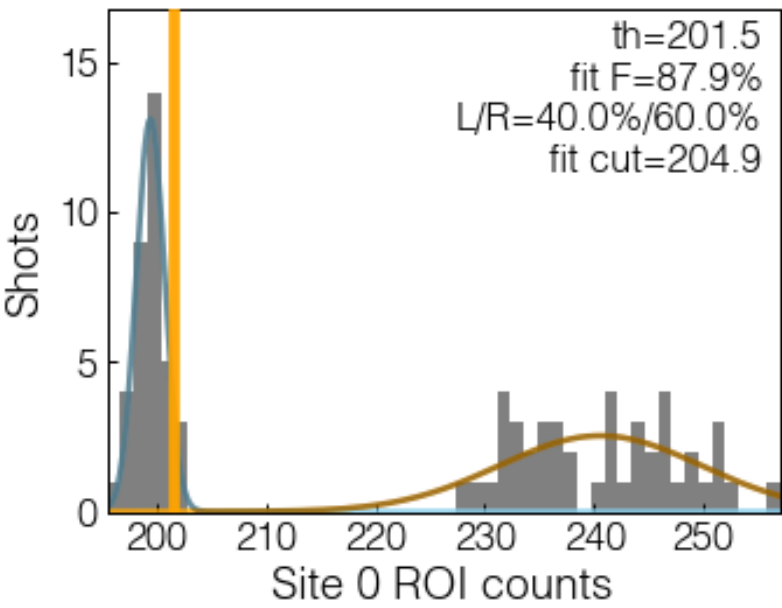


Figure 6.4: threshold calibration 使用已知 ROI 统计 bright/dark count.

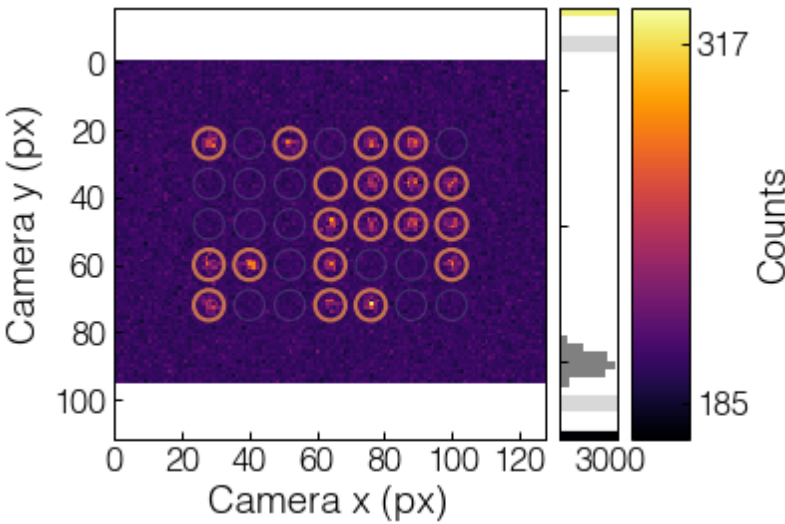


Figure 6.5: detect 输出 occupancy.

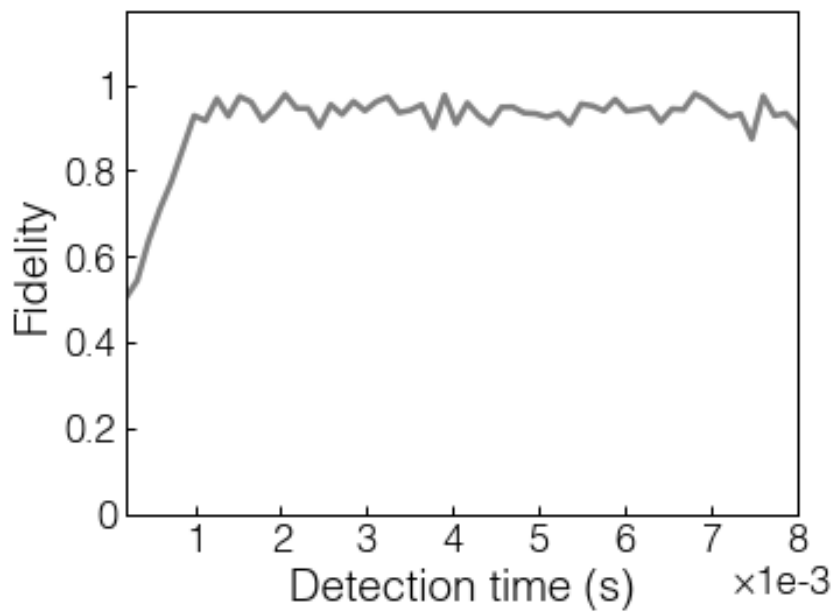


Figure 6.6: detection-time scan 使用同一张 pulse preset, 只改变 `camera_exposure_ns`。

7

常见问题

7.1 pulse 长度是设定两倍

确认 server、GUI 和 notebook 都显示 50 MHz / 20 ns。如果仍按 100 MHz 编译，设定 1 ms 会在 50 MHz 硬件上输出 2 ms。

7.2 On Pulse 没报错但示波器无信号

按顺序检查：

1. FPGA 已 program 当前 `address_switch` bit/LTX。
2. `fpga\run _server.bat -check-config` 指向同一个 bit/LTX/XDC。
3. GUI 打开的 preset 是 `camera_imaging_address_switch.json`。
4. 示波器接的是 `ch09/ch00/ch03/ch11` 对应的 address-switch 物理输出。
5. pulse 是 finite shot 还是 repeat forever；scope trigger 是否设置正确。

7.3 GUI 只显示少数 channel

这是正常的。GUI visible list 是为了降低编辑噪声。上传宽度由 server channel list 决定，missing channel 会自动补 off。

7.4 scan table 不通过

检查每个 timing 参数值是否是 20 ns 的整数倍，检查表达式在每个 scan row 下是否严格递增且不超过 tick width。